S403011 Machine Learning Project: Weather Prediction Team: Forecasting from home

Pierre-Lou Delfante, Alessandro Casorella, Daniel Pinilla Paredes

1 Introduction

The goal of this project is to predict the air temperature in Bern at three forecast horizons ($+12$h, $+24$h, and $+48$h) using various meteorological measurements, notably from the following ten Swiss weather stations: Andermatt, Basel, Davos, Geneva, Interlaken, La Dôle, Lugano, Sion, St. Gallen, and Zermatt. The dataset `train.csv` is provided on Kaggle and contains 7,579 observations and 92 variables, of which 89 are predictors.

Here is an adequate description of the variables:

- For each of the ten stations, several meteorological features were collected: wind speed and gust, global radiation, sea-level pressure, barometric pressure, precipitation, sunshine duration, and air temperature at 2 meters.
- Additionally, there are non-station-specific features: season, hour, and the air temperature in Bern at 2 meters, both current and lagged by 24 hours.
- The three target variables are the air temperature in Bern at forecast horizons of $+12$h, $+24$h, and $+48$h.

We face a multi-horizon regression problem, aiming to predict the same variable at multiple future time points. Modeling requires that predictors be carefully prepared and made analytically usable, and also necessitates a preliminary analysis of the meteorological data. In particular, the presence of missing values, the categorical nature of the `season` variable, and the cyclical structure of `hour` must be carefully handled. Moreover, attention must be paid to the structure of the dataset: the multivariate data from the ten stations require consideration of inter-station redundancy and extreme or unrepresentative values. Measurement errors, as well as genuine meteorological extremes, also need to be accounted for. These issues are addressed through outlier management and strategies to mitigate geographic multicollinearity.

Data scaling, along with the selection and engineering of predictors, will be a key aspect of the modeling process. Furthermore, different machine learning paradigms will be tested to balance flexibility and parsimony, while also considering computational costs, especially for non-parametric models. During model preprocessing, particular care must be taken to prevent data leakage from test subsets or cross-validation folds into the training process.

It is also important to note that the temporal structure of the data is incomplete. In the absence of temporal structure in our data, we may assume that the observations are independent and identically distributed (i.i.d.). Although features such as `hour` and `season` are available, there is no full date-time information or year, preventing a complete reconstruction of chronological order. Therefore, it will not be

possible to create new lagged variables. Nonetheless, by using the already available lagged features (such as the air temperature), we can still derive temporal variations across time, to capture short-term dynamics.

Now that the main challenges of the prediction problem have been presented, the following sections of the report describe, with proper methodological justification, the steps taken in the prediction project:

1. Initial data preprocessing for exploratory analysis
2. Exploratory Data Analysis (EDA)
3. Model-related preprocessing, including data leakage management and feature engineering
4. Model fitting
5. Model comparison and evaluation
6. Results, conclusions, and limitations of the project

The competition metric is the **Mean Absolute Error (MAE)**, a loss function that measures prediction accuracy while being robust to outliers. For a given forecast horizon, the MAE between the true temperatures y_i and the predicted temperatures $\hat{y}_i$ is defined as the average absolute difference between them:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

2 Exploratory Data Analysis

2.1 EDA-Relevant Preprocessing

As previously mentioned, the dataset contains $n = 7,575$ observations with $p = 92$ predictors. In the regression framework, we model $Y = f(X) + \varepsilon$, where Y represents temperature in Bern and $X \in \mathbb{R}^{92}$ contains meteorological variables from 10 weather stations.

All the 3 target variables Y, as well as most features, are numerical (floating-point) variables. However, there are some exceptions:

- `season` is a categorical variable with four levels: Winter, Summer, Spring, and Autumn.
- The variables corresponding to `global_radiation` and `sunshine_duration` are recorded as integer values, reflecting the rounding applied during data collection. However, when these predictors contain missing values for some stations, `pandas` automatically casts the corresponding columns to floating-point type, since the presence of NaN values is incompatible with integer data types.

Most variables have between 1 and 4 missing values. Any row containing missing values cannot be mathematically processed or used for visualization or modeling. For this reason, missing values must be handled prior to any subsequent step.

Before continuing, we emphasize that the following preprocessing stages are primarily EDA-oriented. However, where the two coincide, we anticipate some model-oriented preprocessing, which will simplify the overall project structure.

2.1.1 Handling Missing Values

Targets: Dropping Incomplete Rows For the targets, no imputation is attempted. Training on observations with missing target values would either be impossible for supervised learning or would bias the test error estimate, because for those observations we would have no target to learn from. Therefore, all rows with missing values in any of the target variables are dropped. This slightly reduces the number of observations, but a large sample remains for training.

Since observations with missing targets will not be used during model fitting, we remove them already in the EDA-relevant preprocessing: the primary goal of this step is to prepare the data for modeling.

Features: Imputation For predictors, we adopt imputation rather than row deletion. Dropping rows for missing features would waste valuable data, as none of the observation lines contains more than one NaN (the other values in the row are essential for prediction). In this dataset, no row has more than one missing value, and each feature has at most a few missing entries. Therefore, we perform simple imputation based on the central tendency of the column, which is sufficient given the sparsity of missing values. Imputation with a measure of central tendency minimizes the impact of replacing unknown values on the variable's distribution.

- For the categorical feature `season`, the missing value is imputed with the mode (most frequent category), preserving the empirical distribution.
- For numerical features, we decide between mean and median imputation depending on the presence of outliers. The mean is a central tendency measure that exploits the full distribution, while the median only considers the order of values. However, with many outliers, the mean can be excessively influenced by extreme values. For this reason, features with more than 1% of outliers (conservative, robust arbitrary treshold) are imputed with the median, which is robust to skewness and extreme values, while features with few outliers are imputed with the mean. We use Tukey's rule to define outliers [**?** , Chapter 2].

2.1.2 Outliers and Physical Plausibility

According to our 1% treshold, most features can be considered to contain many outliers. In many cases, outliers indicate measurement errors. But is this the case in our dataset?

For features with many outliers, we programmatically checked whether extreme values are physically plausible. Based on the Python outputs, the ranges for temperature, wind, and radiation align with Swiss meteorological conditions, suggesting they are genuine extremes rather than measurement errors, and are strictly positive and physically plausible. Therefore, these outliers are meaningful for future predictions and should not be removed.

However, outliers can still degrade some models by inflating training error or destabilizing optimization. When minimizing average error, prediction models may overfit to these extreme values, producing estimates inconsistent with the population. For this reason, a more detailed analysis and management of outliers could be considered both in EDA and in model-relevant preprocessing. However, since the competition metric is MAE, which is robust to large deviations, and to avoid excessive plots in EDA, we conclude the outlier analysis here.

2.1.3 Handling Temporal Variables

The `hour` variable is cyclical: hour 23 and hour 0 are consecutive in time but numerically distant. To capture this periodicity, we transform the hour into sine and cosine components:

- `hour_sin` = sin(2 × hour / 24)
- `hour_cos` = cos(2 × hour / 24)

This preserves the circular nature of time while producing two continuous numeric features suitable for modeling. This encoding will be used both for EDA and model fitting. In both cases, computations must account for the cyclical nature of the hour.

Regarding the `season` variable, its categorical nature is problematic: during EDA, this prevents inclusion in correlation matrices or barplots. In future modeling, most algorithms would treat `season` as numeric, potentially introducing false ordinal relationships. In both cases, we replace `season` with three dummy variables corresponding to three seasons (the reference season is left out to avoid multicollinearity).

2.2 Exploratory Analysis

Now that we adequately prepared our dataset, we can start the exploratory analysis. Without claiming to be exhaustive, we aim to identify the main patterns in our data in order to understand the structure of the regression problem and subsequently apply appropriate prediction models.

2.2.1 Correlation structure among features

To assess multicollinearity, we compute the Pearson correlation coefficient ρ between all pairs of features $\mathbf{x}_i, \mathbf{x}_j$:

$$\rho_{i,j} = \frac{\text{cov}(\mathbf{x}_i, \mathbf{x}_j)}{\sigma_i \sigma_j} \tag{2.1}$$

Due to the symmetry of the correlation matrix $\mathbf{R}$, we focus on the upper triangular matrix $\mathbf{R}_u$ where $j > i$, to derive non-redundant statistics.

High correlation between predictors can indicate multicollinearity, which inflates the variance of Ordinary Least Squares estimates, making linear models unstable and sensitive to small changes in the data. That motivates the use of regularization predictive methods.

```
corr_matrix = dataset[feature_columns].corr()
upper_triangle = corr_matrix.values[np.triu_indices_from(
    corr_matrix.values, k=1)]
```

But do we actually face multicollinearity? After processing the feature set, the following global correlation statistics were observed:

Extreme Correlations: Range from $\rho_{\min} = -0.124$ to $\rho_{\max} = 0.988$, suggesting that most of the features are positively correlated among them.

Absolute Minimum: The weakest linear dependency found was $|\rho|_{\min} = 0.001$.

Distribution: The mean pairwise correlation $\bar{\rho} = 0.35$ with $\sigma_\rho = 0.15$.

To identify potential multicollinearity, we look for feature pairs with absolute correlation above 0.85. This threshold is arbitrarily chosen to flag strong linear dependencies.

Geographical multicollinearity:

A primary source of redundancy comes from geographical proximity; meteorological variables recorded across different stations show strong spatial correlation. For example, `air_temp_2m_ANT`, `air_temp_2m_BAS`, etc. are strongly related.

To analyze this, we define a set of core meteorological features $\mathcal{V}$:

```
key_variables = [
    "air_temp_2m", "rel_humidity_2m", "wind_speed", "wind_gust",
    "global_radiation", "pressure_sea_level", "pressure_barometric",
    "precipitation", "sunshine_duration"
]
```

For each key variable $v \in \mathcal{V}$, we construct a feature group $\mathcal{G}_v$ containing all station-specific columns. The screening process follows a structured pipeline:

1. **Intra-group Correlation:** Compute the pairwise correlation matrix $\mathbf{R}_v$ for all $\mathbf{x} \in \mathcal{G}_v$.
2. **High-Correlation Filtering:** A group is flagged only if $\exists \rho_{i,j} \in \mathbf{R}_v$ such that $|\rho_{i,j}| > 0.85$ (excluding the diagonal) [**?** , Slide 3]
3. **Sparsity Masking:** To prioritize critical redundancies, we apply a mask:

$$f(\rho) = \begin{cases} \rho & \text{if } |\rho| \geq 0.85 \\ \text{NaN} & \text{otherwise} \end{cases} \tag{2.2}$$

4. **Visual Encoding:** Generate heatmaps for the flagged groups to identify clusters of stations with near-identical weather patterns.

All these heatmaps are arranged in a grid and saved as `geographical_correlations.pdf` (Figure 2.1).

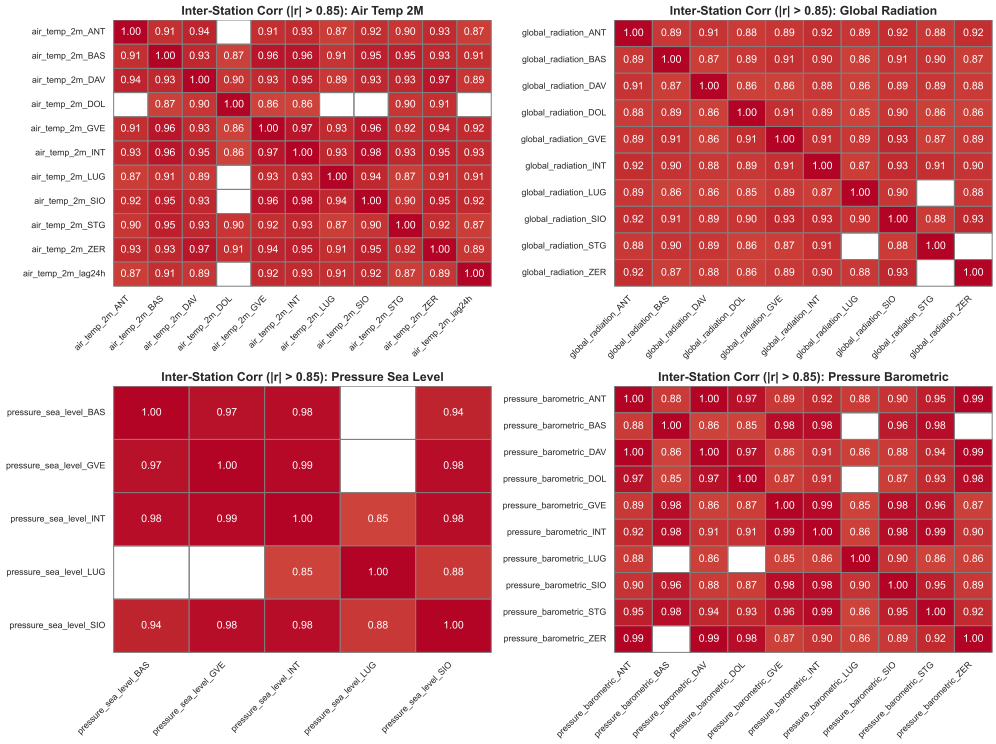

Figure 2.1: Inter-station correlation matrices for key meteorological variables (absolute correlations > 0.85).

These plots highlight strong spatial dependencies: some stations behave very similarly for certain variables (e.g., temperatures in nearby regions).

Structural multicollinearity:

Beyond geography, there may be structural correlations between different meteorological variables. To focus on these, we:

- identify station pairs that are already part of the geographical groups;
- construct new groups of variables that avoid placing highly correlated station pairs together;
- split these groups into blocks of at most 20 variables for readability.

For each block, we compute the structural correlation matrix, keep only entries with absolute value above 0.85, and plot the result. All blocks are saved in `structural_correlations.pdf` (Figure 2.2).

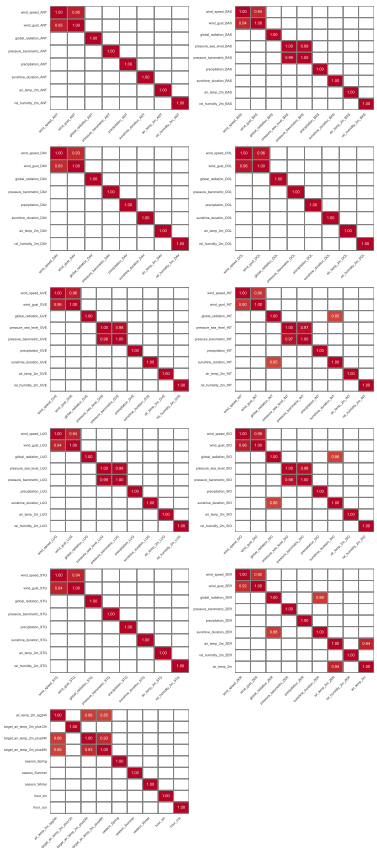

Figure 2.2: Blocks of structural correlations among features (absolute correlations > 0.85), excluding purely geographical duplicates.

These visualizations provide a compact overview of remaining multicollinearity due to structural relationships between variables.

In almost all stations, we observe strong multicollinearity between `wind_gust` and `wind_speed`, as well as between `pressure_barometric` and `pressure_sea_level`. These are two pairs of very similar meteorological variables, which alone can account for almost the entirety of the structural multicollinearity in our dataset.

2.2.2 Correlation between features and targets

Feature-target correlation:

We then study how each feature correlates with each of the three targets. For a given target T, we compute:

```
correlations = dataset[features].corrwith(dataset[T])
```

We keep only the features whose absolute correlation exceeds a certain threshold:

- 0.3 for the $+12$h horizon, to include a broader set of moderately informative features for short-term prediction, since correlations between features and target are less stable at short horizons.
- 0.4 for the $+24$h and $+48$h horizons, to focus on the most informative features for longer-term predictions and avoid including too many weakly correlated features. At these longer horizons, the effects of features become clearer and correlations with the target are more pronounced.

The selected features are stored in three lists: `main_features_12h`, `main_features_24h`, and `main_features_48h`. We then generate barplots of these correlations for each horizon, coloring positive correlations in green and negative correlations in red. The three barplots are combined into a single figure saved as `target_correlations_thresholded.png` (Figure 2.3).

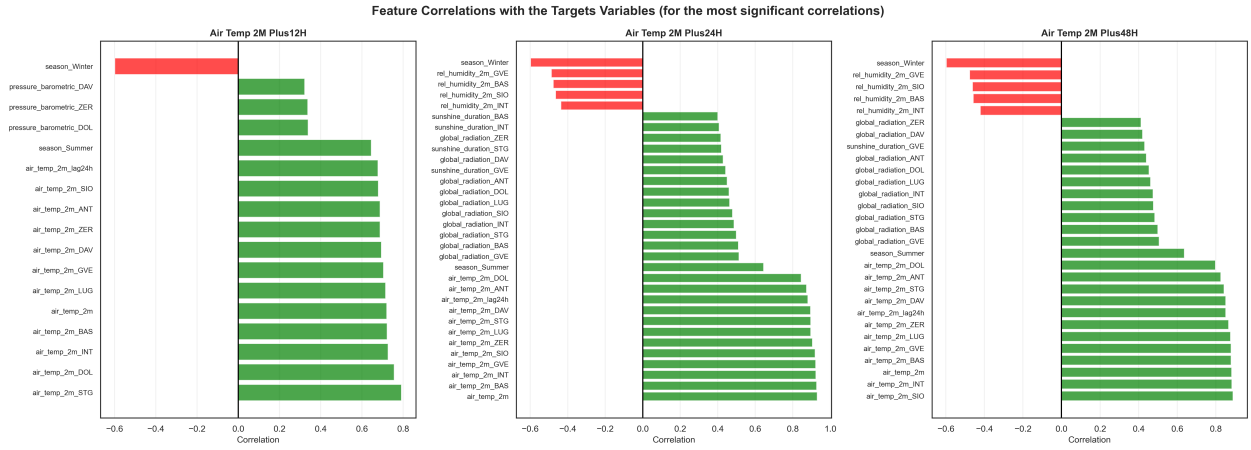

Figure 2.3: Most strongly correlated features for each target horizon (absolute correlation above the chosen thresholds).

These plots identify the most informative features for each horizon and motivate the reduced feature sets used later in the linear models.

Overall, we observe that, for all three targets, the most important features are those related to temperature measurements across different stations, as well as the categorical feature corresponding to Winter, highlighting the influence of local temperature patterns and seasonal effects on predictive performance.

It is interesting to note that the time of day does not have a substantial impact on the targets; without an underlying temporal structure, the cyclic nature of the hour does not manifest in the data.

Station-target correlation:

To better understand the spatial structure, we analyze, for each key meteorological variable, the correlation between measurements from different stations and the three targets. The resulting correlations are arranged in a matrix where rows correspond to stations (ordered from top to bottom by decreasing mean absolute correlation) and columns correspond to the target horizons.

Each matrix is visualized as a heatmap and saved in `targ_stat_corr.pdf` (Figure 2.4).

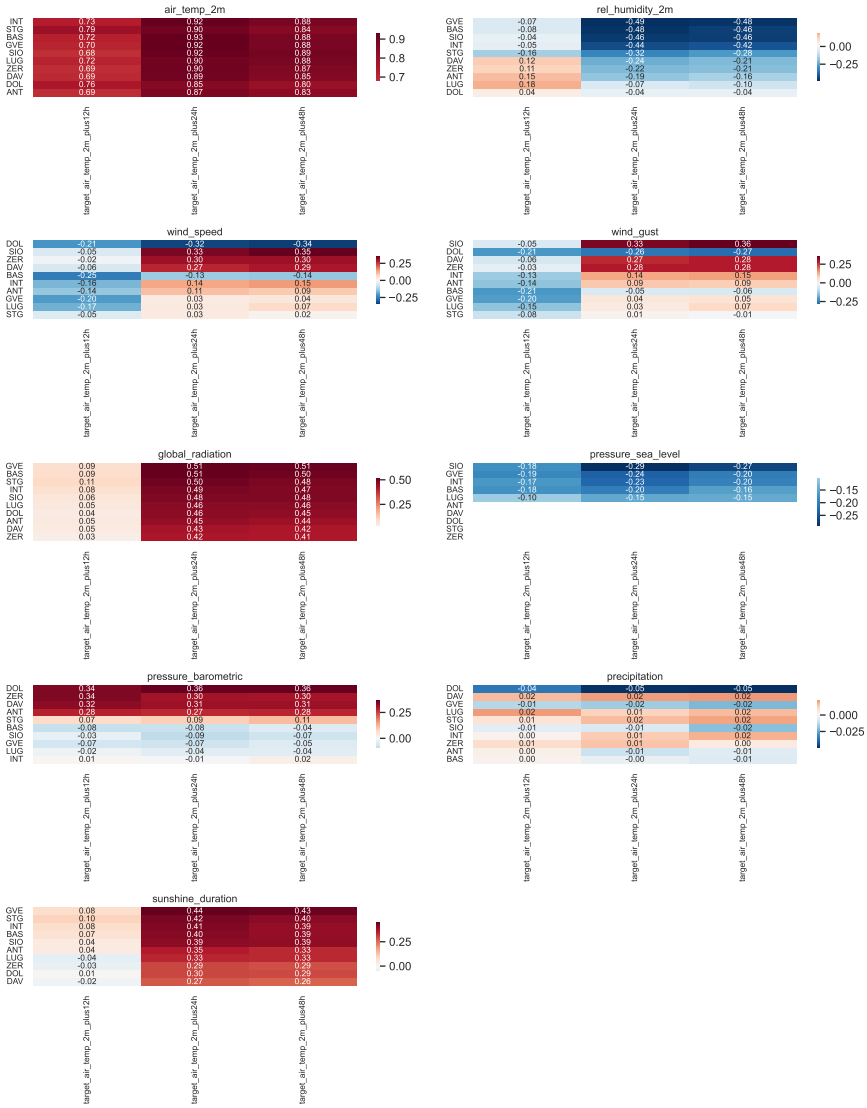

Figure 2.4: Correlation between targets and station-specific measurements for each key meteorological variable. Stations are ordered by decreasing mean absolute correlation.

By examining the heatmaps for the different key variables, we observe which stations contribute more predictive information for each measurement and horizon. However, no station appears to be strictly more important than the others across all variables.

2.2.3 Distribution analysis of key predictors

Focusing on the $+24$h horizon (the main competition target), we select a subset of predictors, `reduced_features`, constructed as follows: for each pair of features with very high correlation (absolute correlation above 0.85), we retain the one whose average correlation with the three targets is largest. This is a pedagogical alternative to classical VIF-based selection, suitable for our EDA, although it does not account for non-linear multicollinearity. This feature reduction can help us focus on the main, non-redundant predictors for the rest of the EDA.

For these `reduced_features`, we display the distribution of those variables whose absolute correlation with `target_air_temp_2m_plus24h` exceeds 0.4. For each selected feature, we plot a kernel density estimate (KDE) to visualize its distribution.

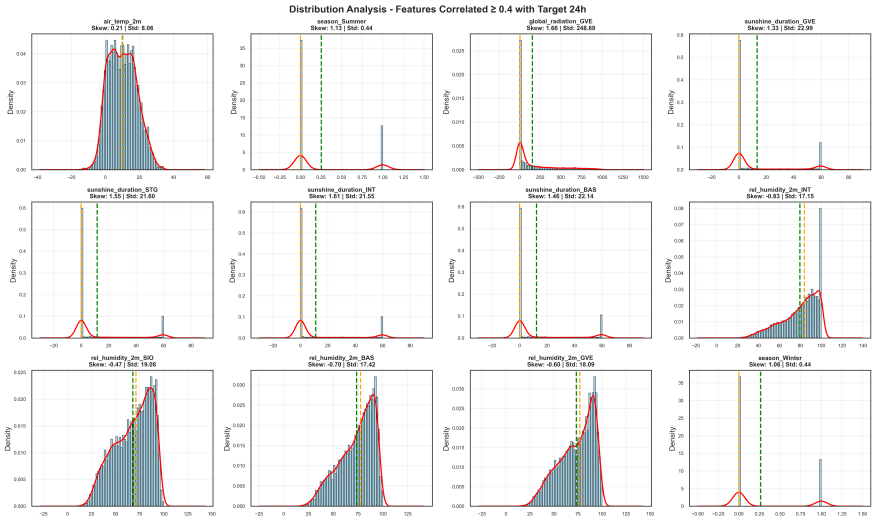

Figure 2.5: KDE distributions of the most strongly correlated features with the $+24$h target (absolute correlation ≥ 0.4).

Several variables, particularly those related to humidity, are negatively skewed. This indicates an overall climate characterized by high humidity, with only occasional dry periods. The scarcity of extreme values suggests that the linear coefficients for these features should remain generally stable. For other features, notably sunshine duration, we observe a bimodal distribution. This pattern is likely due to the coexistence of two predominant sunshine regimes: the winter regime, corresponding to the left peak, and the summer regime, corresponding to the right peak. In this context, the bimodality suggests that the interaction between sunshine duration and season could provide an interesting avenue for analysis within the framework of linear regression.

2.2.4 Target exploration

From the target distribution analysis, we observe that all three targets are approximately normally distributed and continuous. Although normality is not a requirement for regression, even linear regression,

the continuity of the targets confirms that regression methods are appropriate for modeling these variables, as opposed to classification approaches. For space considerations, the corresponding plots are not shown here, but they can be fully reproduced using the code provided.

Our Python-based exploratory analysis also reports basic descriptive statistics for each target variable, including the mean, standard deviation, extreme values, and selected quantiles. Across the three targets, the mean and standard error are remarkably similar, fluctuating around $10\,^{\circ}\mathrm{C}$ and $8\,^{\circ}\mathrm{C}$ respectively. Likewise, the distributions' extremes and quantiles are very close across horizons.

This similarity is unsurprising, as the three targets represent the same underlying variable observed at different time horizons. Consequently, a strong correlation between targets is expected. This high degree of redundancy across dependent variables reduces the potential benefits of a multi-output regression framework, particularly from an interpretability perspective.

The comparable standard deviation across targets indicates a consistent level of variability around the mean. Importantly, the targets are not quasi-constant and exhibit sufficient dispersion to allow the regression model to identify meaningful associations with the explanatory variables. In other words, the observed variability provides adequate statistical leverage to investigate the dynamic relationship between the targets and the selected features.

3 Methodology: training protocol and leakage prevention

We aim to perform supervised forecasting at three horizons (12h, 24h, 48h). For each horizon $h \in \{12, 24, 48\}$, the data define a regression dataset

$$\mathcal{D}^{(h)} = \{(x_i, y_i^{(h)})\}_{i=1}^{n},$$

where x_i is a vector of meteorological predictors (station-level and temporal variables) and $y_i^{(h)}$ is the temperature target at horizon h. As justified in the target exploration, we prefer treating these horizons as *three distinct learning problems* sharing the same protocol: the same splitting logic, the same preprocessing discipline, and the same evaluation metric. This strict homogeneity allows us to interpret performance differences as genuine model effects rather than artifacts of data handling.

A core methodological constraint is **leakage prevention**. Any transformation that uses information from the distribution of predictors (imputation values, scaling parameters, dummy coding, etc.) must be learned *only* from training data and then applied to validation/test data. In the code, this process is enforced by `scikit-learn` `Pipeline`s: preprocessing is not performed "once and for all" on the full dataset; instead, it becomes part of the estimator object and is refit whenever the estimator is refit.

3.1 Train/test split

We begin by constructing a single held-out test split used as a final sanity check across all models and all horizons. Concretely, the script builds (X, y), then applies a random split with a fixed seed for reproducibility:

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, shuffle=True, random_state=42
)
```

This held-out test split is not used for hyperparameter selection. It only appears at the end of each model block, when the final fitted pipeline produces predictions $\hat{y}_{test}$ and we compute a hold-out MAE. This separation matches the "train/tune/evaluate" discipline emphasized in the course: the test set remains a genuinely unseen check.

3.2 Leakage-safe preprocessing with `Pipeline`

All models share a common structure: a preprocessing operator $g(\cdot)$ followed by a predictor $f_\theta(\cdot)$. So predictions take the form

$$\hat{y} = f_\theta\big(g(x)\big).$$

In the code, this composition is implemented via:

```
pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", <estimator>)
])
```

Preprocessing is handled by a dedicated utility function `make_preprocessor(features, numeric_cols, scale_numeric=...)`. Its role is twofold: (i) ensure that every model uses the same feature-selection interface (the `features_to_use` list), and (ii) apply transformations in a leakage-safe, column-wise manner through a `ColumnTransformer`. We now detail, for each feature type, on the specific transformation we apply:

Numeric block (median imputation + optional robust scaling). For numeric predictors (excluding special variables `hour` and `season`), the preprocessor applies:

- `SimpleImputer(strategy="median")`, consistent with the EDA discussion of robustness to skewness/outliers. Unlike the EDA-oriented preprocessing, we do not rely on Tukey's rule to arbitrate between mean and median imputation at the modeling stage. Instead, we adopt a unified imputation strategy, as `SimpleImputer` integrates more naturally within a `FunctionTransformer`-based preprocessing pipeline.
- `RobustScaler()` when `scale_numeric=True`. Robust scaling centers/scales using quantiles, which is coherent with heavy-tailed predictors in meteorological data and avoids overly reacting to extremes.

This block is activated for linear models (especially Ridge), and disabled for tree ensembles where scaling is not required. Indeed, heterogeneous feature scales bias the interpretation of linear coefficients and interfere with the estimation of regularized models, such as Ridge or Lasso regression.

Hour block (cyclical encoding). The variable `hour` is treated as cyclical rather than ordinal. The preprocessor uses a `FunctionTransformer(hour_to_cyclical)` after median imputation, producing sine/cosine components. This reproduces the ML2 logic: hour 23 and hour 0 should be close in representation, not distant.

Season block (categorical encoding). The variable `season` is handled as categorical. The code imputes the most frequent category and applies a dedicated one-hot encoder via `_make_onehot_encoder()`. This avoids introducing artificial ordinality and yields a consistent set of dummy variables across splits (with unknown categories handled safely).

Strict column control. The `ColumnTransformer` is constructed with `remainder="drop"`, meaning that only explicitly listed columns flow into the model. This is important in practice: the model input is completely determined by `features_to_use`, which ensures that all comparisons across models/horizons are made on well-defined feature sets.

3.3 Evaluation metric and resampling

The competition metric is the Mean Absolute Error (MAE), so we align both tuning and reporting with:

$$\mathrm{MAE}(y, \hat{y}) \;=\; \frac{1}{m}\sum_{i=1}^{m} |y_i - \hat{y}_i|.$$

MAE is expressed in degrees Celsius and remains stable under occasional large deviations, which is consistent with the outlier discussion in the EDA section.

3.3.1 Cross-validation object

To estimate out-of-sample performance on the training split, we use K-fold cross-validation with shuffling and a fixed seed:

```
cv_folds = KFold(n_splits=10, shuffle=True, random_state=42)
```

The choice $K = 10$ follows the standard bias–variance compromise recommended in the course.

3.3.2 A single evaluation routine for comparability

A key design choice in the script is to centralize evaluation in `evaluate_model`. This function does two things:

1. runs `cross_validate` with `scoring="neg_mean_absolute_error"` and `return_train_score=True` to obtain both CV training and CV validation MAE;
2. refits the pipeline on the full training split and produces predictions on the held-out test set.

In code:

```
cv_results = cross_validate(
    pipeline, X_tr, y_tr,
    scoring="neg_mean_absolute_error",
    cv=cv,
    return_train_score=True,
    n_jobs=-1
)
mae_train_cv = -cv_results["train_score"].mean()
mae_val_cv   = -cv_results["test_score"].mean()

pipeline.fit(X_tr, y_tr)
y_pred_te = pipeline.predict(X_te)
```

This structure mirrors the ML2 reporting convention: for each model/horizon we retain (i) CV train MAE (under/overfitting indicator), (ii) CV validation MAE (primary comparison metric), and (iii) test MAE (final sanity check).

3.3.3 Internal vs external resampling (hyperparameters vs reporting)

Some models perform an *internal* resampling loop for hyperparameter selection (notably Ridge via `GridSearchCV`). The script keeps this separate from the *external* CV used for comparison. Concretely, `GridSearchCV` identifies the best hyperparameters, and `evaluate_model` is then applied to the selected estimator to report CV MAE under the same evaluation function as the other models. This ensures that "CV MAE" is computed in a comparable way across all methods.

4 Feature engineering

Given the high degree of multicollinearity in the dataset, particularly arising from geographical redundancy, we deliberately avoid constructing interaction features, as they could further amplify collinearity and destabilize the estimation of linear coefficients. Instead, we define an augmented feature set X_2 that leverages the intrinsic properties of our data, creating structured, physically meaningful summaries. These higher-level signals are designed to be informative for both linear and non-parametric models without introducing additional redundancy.

In the script, all engineered features are built by `add_fe_features_basic`, which composes two deterministic row-wise transformations:

```
def add_fe_features_basic(X_):
    Xf = X_.copy()
    Xf = add_station_aggregates(Xf)
    Xf = add_temp_delta_24h(Xf)
    return Xf
```

4.1 Station aggregation features

The dataset contains many station-specific variables of the form `<base>_<station>`. EDA highlighted strong geographical multicollinearity: stations that are close tend to move together. The function `add_station_aggregates` operationalizes this observation by building row-wise summaries across the station dimension.

Construction in code. The function first detects station columns via `_station_suffix` and groups them by base variable `_base_var`. For each base variable with at least two available stations, it computes:

$$\mu_t = \frac{1}{S}\sum_{s=1}^{S} x_{s,t}, \qquad \sigma_t = \sqrt{\frac{1}{S}\sum_{s=1}^{S}(x_{s,t}-\mu_t)^2},$$

as well as min/max and range. These appear as new columns:

```
Xf[f"{base}__mean_stations"]   = vals.mean(axis=1)
Xf[f"{base}__std_stations"]    = vals.std(axis=1)
Xf[f"{base}__min_stations"]    = vals.min(axis=1)
Xf[f"{base}__max_stations"]    = vals.max(axis=1)
Xf[f"{base}__range_stations"] = Xf[f"{base}__max_stations"] - Xf[f"{base}
    __min_stations"]
```

Interpretation. These aggregates act as "spatial state descriptors": they summarize the general level (mean), the spatial heterogeneity (std), and extremes (min/max) at time t. In a forecasting context, this is practically useful because it compresses a high-dimensional block of redundant station columns into signals that remain interpretable and stable. Importantly, given the high degree of geographical multicollinearity, these aggregations introduce minimal bias while reducing the variance of predictions. Tree ensembles can further exploit these summaries with split rules such as "high mean radiation but low dispersion", capturing regimes that would otherwise require deeper trees or more data.

4.2 Temperature dynamics: 24-hour differences

Weather series exhibit strong diurnal persistence and day-to-day transitions. The script encodes this via 24-hour temperature differences, constructed whenever both contemporaneous and lag-24h temperature columns exist.

Construction in code. For each station s:

$$\Delta T^{(24)}_{s,t} = T_{s,t} - T_{s,t-24h}.$$

This is implemented as:

```
t_now = f"air_temp_2m_{st}"
t_lag = f"air_temp_2m_lag24h_{st}"
Xf[f"air_temp_2m__delta24h_{st}"] = Xf[t_now] - Xf[t_lag]
```

The same construction is also applied to the Bern (non-suffixed) columns when present.

Interpretation. $\Delta T^{(24)}$ behaves like a local trend indicator: it distinguishes "warming relative to yesterday" from "cooling relative to yesterday". This is particularly relevant for short horizons (12h/24h), where the near-term trajectory of the atmosphere often matters as much as absolute levels.

4.3 Leakage-safe application to the train/test split

A key point is that FE should not change the evaluation protocol. The script therefore constructs X_2 once, then reuses the *same* train/test indices:

```
X2 = add_fe_features_basic(X)
y2 = y.copy()

X2_train = X2.loc[X_train.index]
X2_test  = X2.loc[X_test.index]
y2_train = y_train
y2_test  = y_test
```

Methodologically, this is safe because the engineered features are deterministic row-wise transformations: they do not estimate global parameters from the dataset (no learned scaling, no PCA, no target encoding). Practically, reusing indices guarantees that improvements measured under FE reflect representation gains rather than a different split composition.

5 Models fitting and hyperparameter strategy

This section explains the modelling blocks implemented in the Python script and the logic behind the tuning choices. All models follow the same high-level structure: a leakage-safe preprocessing stage followed by a regression estimator, encapsulated in a `Pipeline`. This design makes the training protocol homogeneous across algorithms and horizons, since the same object governs (i) preprocessing fit on training folds and (ii) transformation/prediction on validation and test data. For simplicity reasons, following paragraphs will focus on the 24h target.

A second organizing principle is that the project compares models under the same MAE-based protocol: hyperparameter selection is performed with a model-appropriate strategy, while performance reporting is standardized using the shared `evaluate_model` routine (CV train MAE, CV validation MAE, refit-train MAE, and hold-out predictions).

5.1 Baseline linear regression

The first model is a linear regression baseline (`LinearRegression`), used as a transparent reference point. Predictions take the form

$$\hat{y} = \beta_0 + \sum_{j=1}^{p} \beta_j \, z_j,$$

where $z = g(x)$ denotes the preprocessed feature representation. This baseline provides a simple benchmark for MAE, yielding a training MAE of approximately 2.07.

Implementation details. The model restricts features using EDA-driven filters: (i) `main_features_12h/24h/48h` (features highly correlated with each target), and (ii) `reduced_features_model` obtained by applying `reduce_multicollinearity` with a threshold of 0.95. The intersection of these filters per horizon forms `feature_map_linear[target]`, ensuring numerical stability and easier interpretability.

5.2 Ridge regression (regularized linear model)

Ridge regression addresses multicollinearity by adding ℓ_2 shrinkage:

$$\hat{\beta}(\alpha) = \arg\min_{\beta} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \beta^\top z_i)^2 + \alpha \|\beta\|_2^2 \right\}.$$

By shrinking correlated coefficients, Ridge reduces variance and stabilizes predictions. Training on all available predictors, the model achieves a lower training MAE of roughly 1.93.

Implementation details. Scaling is applied to numeric features (`scale_numeric=True`) so that the regularization strength α is comparable across variables with different units. Hyperparameter α is selected through grid search with a logarithmic scale, allowing CV to determine the optimal shrinkage per horizon.

5.3 Random Forest (bagging-based ensemble)

Random Forest provides a flexible non-parametric alternative that captures non-linearities and interactions. By averaging many deep trees, it reduces variance relative to a single tree while remaining highly expressive. The training MAE is approximately 0.69, reflecting strong predictive performance, though the model can be prone to overfitting if not properly regularized.

Implementation details. The script uses OOB estimates to select `max_features` and `min_samples_leaf`, training an initial 500 trees for tuning, then refitting the final forest with 2000 trees. Scaling is not applied (`scale_numeric=False`) since tree splits are invariant to monotone feature transforms.

5.4 Gradient Boosting with early stopping

Gradient Boosting sequentially fits trees to reduce prediction error. Using `loss="absolute_error"`, it directly optimizes MAE. Early stopping determines the effective ensemble size B^*, with a subsequent refit to finalize the model. This procedure yields a training MAE of approximately 1.29.

Interpretation. Compared to the raw Random Forest, boosting is slightly less flexible but more stable, controlling variance through early stopping and additive shrinkage. The lower MAE compared to linear models reflects its ability to capture non-linear effects without explicit feature interactions.

5.5 Gradient Boosting on engineered features (GB + FE)

Applying the same boosting routine to the engineered feature set $X2$ leverages inter-station aggregates and 24-hour temperature differences. Aggregation reduces dimensionality and stabilizes variance, while retaining informative signals for tree-based learners. The training MAE is around 1.59, slightly higher than the unaggregated GB model (1.29) due to the minor bias introduced by the station-level aggregates, but predictions are more robust and less sensitive to multicollinearity.

Implementation details. Only base columns without station suffix and engineered columns are retained, while redundant raw station features are dropped. This allows boosting to exploit high-level dynamics and spatial summaries without inflating dimensionality.

6 Results: model selection and diagnostics

6.1 Model selection criterion: validation MAE under cross-validation

Our primary selection metric is the **cross-validation validation MAE** (CV val MAE). This choice follows the standard principle used throughout the project: *CV validation MAE is used for model comparison*, while the hold-out test split is reserved for a final sanity check and error diagnostics. The CV train MAE is reported only to diagnose under/overfitting through the train–validation gap.

Table 6.1 summarizes the results for the main model families (OLS, Ridge, Random Forest, Gradient Boosting). The ranking is consistent across horizons: (i) linear OLS provides a transparent baseline but suffers from high bias; (ii) Ridge improves stability under multicollinearity but remains limited by linearity; (iii) tree ensembles (RF, GB) achieve the best validation MAE thanks to their ability to capture non-linearities and interactions that are plausible in meteorology.

6.2 Selected model: Gradient Boosting (early stopping + refit)

Based on Table 6.1, we select **Gradient Boosting (GB)** as final model family because it achieves the best (or near-best) **CV validation MAE** while keeping a controlled generalization gap. Conceptually, boosting builds an additive model by sequentially correcting residual errors. This typically reduces bias

Table 6.1: Model comparison using CV validation MAE (primary selection metric) and hold-out MAE.

Model	**+12h**		**+24h**		**+48h**	
OLS	CV val: (...)	Hold-out: 3.38	CV val: (...)	Hold-out: 2.07	CV val: (...)	Hold-out: 2.60
Ridge	CV val: (...)	Hold-out: 1.96	CV val: (...)	Hold-out: 2.00	CV val: (...)	Hold-out: 2.55
RF	CV val: (...)	Hold-out: 1.84	CV val: (...)	Hold-out: 1.87	CV val: (...)	Hold-out: 2.40
GB	CV val: (...)	Hold-out: 1.63	CV val: (...)	Hold-out: 1.90	CV val: (...)	Hold-out: 2.41

relative to bagging methods, but requires strong regularization. In our implementation, regularization is mainly ensured by: (i) **shallow trees and leaf constraints**, and (ii) **early stopping**, which automatically selects the effective number of trees $B^\star$ and prevents unnecessary iterations. Importantly, GB is trained with an MAE-aligned objective (absolute-error loss), matching the competition metric.

6.3 Hold-out diagnostic of the selected model (GB)

We now diagnose GB using the hold-out test split (kept separate from model selection). The hold-out MAE values are:

- **+12h**: MAE = 1.63
- **+24h**: MAE = 1.90
- **+48h**: MAE = 2.41

Horizon difficulty. Performance degrades as the horizon increases: +48h is the hardest target. This is expected because the predictive content of contemporaneous meteorological measurements decays with lead time, and uncertainty compounds over longer horizons.

Bias and residual shape. The residual histograms for GB are approximately centered around zero for all horizons, indicating limited global bias. However, dispersion increases with the horizon, consistent with greater irreducible uncertainty at +48h. The presence of heavier tails suggests that rare regimes (fast transitions, extremes, or heterogeneous spatial states) remain harder to capture with a single global model.

6.4 Why GB over RF despite close hold-out MAE on some horizons

Random Forest is competitive on hold-out MAE (notably close to GB at +24h and +48h), but diagnostics reveal stronger overfitting. Indeed, RF achieves extremely small refit-train MAE (e.g., around 0.68 at +12h/+24h and 0.91 at +48h), while its hold-out MAE remains substantially larger (1.84, 1.87, 2.40). This large train–test gap is a classic variance/overfitting signature for deep trees, even under bagging.

By contrast, GB shows a smaller train–test gap (e.g., refit-train MAE around 1.14 / 1.29 / 2.05 versus hold-out 1.63 / 1.90 / 2.41), which is consistent with early stopping acting as a strong complexity

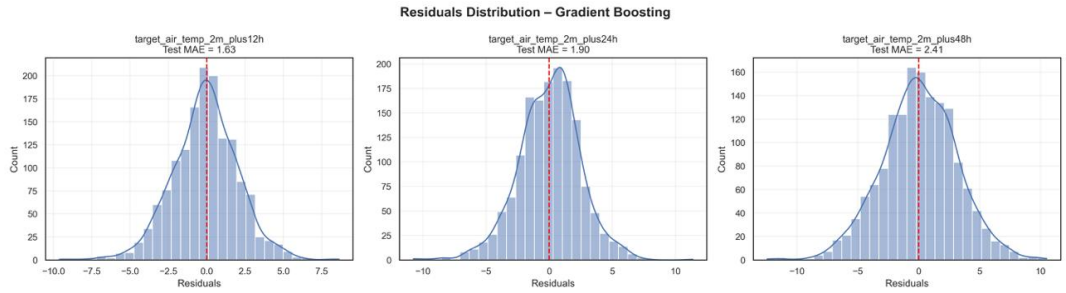

Figure 6.1: Hold-out residual distributions for Gradient Boosting (GB), with three panels corresponding to +12h, +24h and +48h. Residuals are approximately centered around zero across horizons (limited global bias), while dispersion and tail thickness increase with lead time, consistent with higher uncertainty at +48h.

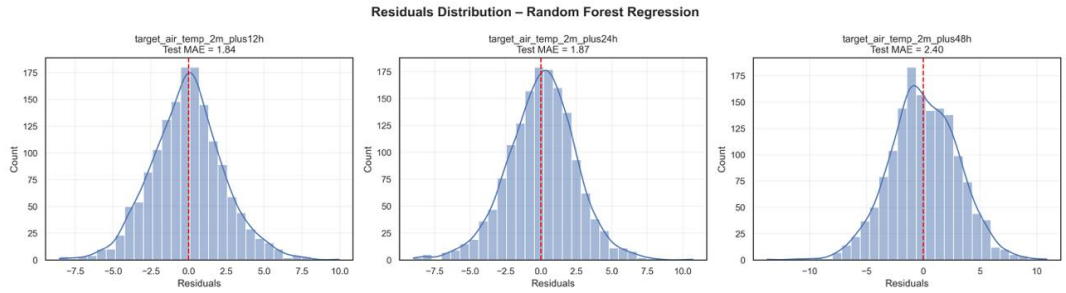

Figure 6.2: Hold-out residual distributions for Random Forest (RF), with three panels corresponding to +12h, +24h and +48h. Despite competitive hold-out MAE, RF shows a stronger train–test gap (variance signature), which motivates preferring GB with early stopping for more controlled generalization.

controller. Therefore, even when hold-out MAE is close, GB is preferred for its better-controlled generalization and its metric-aligned learning objective.

Practical recommendation. For the Kaggle submission target (+24h), we recommend using the **GB early-stopping refit pipeline** as the default final predictor. A natural extension (if time permits) is to exploit feature engineering specifically designed for tree ensembles (station aggregates and temperature dynamics), which can enrich the representation with spatial summaries and short-run trends and may further reduce validation MAE.

6.5 Conclusion

Across horizons, the results confirm that flexible non-linear learners are better suited to meteorological forecasting than purely linear specifications: tree ensembles systematically dominate OLS and Ridge in CV validation MAE, consistent with the presence of non-linear effects and interactions in atmospheric variables. Among the candidates, Gradient Boosting (GB) offers the best bias–variance trade-off when

regularized by early stopping, yielding strong generalization and a controlled train–test gap. On the hold-out set, performance degrades with lead time—a natural consequence of increasing uncertainty—with MAE values of 1.63 ($+12$h), 1.90 ($+24$h), and 2.41 ($+48$h). Random Forest remains competitive, especially at $+24$h and $+48$h, but exhibits a larger train–test gap, indicating higher variance relative to GB. Therefore, for the Kaggle target horizon ($+24$h), we recommend the GB early-stopping refit pipeline as the default final predictor. Feature engineering based on spatial aggregates and 24-hour temperature dynamics remains a promising extension, as it can compress geographic redundancy into interpretable signals and potentially improve stability.

6.6 Limitations and threats to validity

Two limitations should be highlighted. First, the dataset lacks a full timestamp, preventing time-ordered splits and blocked cross-validation; as a result, our evaluation relies on an i.i.d. assumption with random splitting, which may slightly overestimate performance compared to a realistic chronological setting. Second, early stopping introduces an additional selection step (the effective number of boosting iterations). If the stopping iteration is tuned outside the outer CV loop (or reused across folds), CV estimates can become mildly optimistic relative to a fully nested procedure. A stricter approach would perform early stopping within each CV fold (or use nested CV for all tuning decisions), at the cost of additional computation. These caveats are unlikely to overturn large performance differences, but they matter when competing models are very close in MAE.

...

References

[1] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Springer. https://www.statlearning.com/

[2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer. https://web.stanford.edu/~hastie/ElemStatLearn/

[3] Engelke, S. (2025). *Machine Learning Course*. University of Geneva. https://www.unige.ch/formations/cours/machine-learning/

AI declaration : AI tools were used as a support throughout this project, mainly to make the work cleaner, clearer and more efficient, but not to replace our own decisions or experiments. AI helped us debug Python code when we ran into errors in the preprocessing and modelling pipelines, for example by suggesting how to fix issues with feature selection, cross-validation setup, and data leakage. It also helped us rewrite parts of the code to be more efficient in terms of computation time, for instance by choosing more reasonable grids for hyperparameter tuning and by avoiding redundant cross-validation runs when reusing the best parameters. We also used AI to help with the report writing in LaTeX. This included suggesting clearer English formulations, simplifying long paragraphs, and checking that the terminology for models, features and targets was used consistently across sections. AI was useful for drafting short explanations of standard machine learning concepts (such as train–test split, cross-validation or regularization) based on the course material, which we then edited to fit our specific models and results. In addition, AI assisted in organizing and formatting the bibliography. It helped identify which lecture chapters and slides were relevant to each part of the project and suggested consistent BibTeX entries for the course notes and textbooks. We then checked and adjusted these references to match the official material. Finally, all modelling choices (which models to use, which hyperparameters to tune, how to select features) and all numerical results come from our own experiments on the dataset. AI was used to speed up debugging, improve text quality and suggest more efficient implementations, but the design and interpretation of the models remain our own work.